LMS Adaptive Filtering (C++ & Python)

Introduction

In audio engineering, achieving pristine sound quality is an intricate task influenced by multiple variables. Factors such as the choice of microphone, the environment of recording, the recording device's quality, and the post-production processes all play pivotal roles in determining the accuracy of a sound recording. A meticulous approach to each of these components is essential to attain a high calibre recording that resonates with clarity and precision.

In the context of this exercise, we were presented with authentic sound files ex- tracted from actual recordings intended for CD production. The specific challenge at hand was the presence of sound interference, where the dominant sound of trumpets inadvertently bled into the vibraphone track, overshadowing its original sound. This kind of audio bleed, a common occurrence in multi-instrument recordings, poses a significant obstacle to achieving a clean and distinct audio output. To address this issue, we leveraged the classical technique of *Adaptive Filtering*. Adaptive filtering is a signal processing method that dynamically adjusts filter coefficients to optimize a predefined objective function. While a Python-based filtering library was initially provided for this task, our primary objective was to reimplement this algorithm in C++ to potentially enhance computational efficiency and achieve better filtering outcomes.

The goal of this program was to develop a C++ program capable of processing and filtering audio files in the .wav format using specialized external libraries. The WAV (Waveform Audio File format) files, renowned for their high-fidelity and uncompressed nature, store audio data as numerical representations of waveform shapes and intensities. These numerical representa- tions can be manipulated through arithmetic operations to modify the audio output. In our case, the adaptive filtering algorithm manipulated these numerical representations to cleanse the vibraphone track, effectively removing the unwanted trumpet sound bleed.

The program procedure was structured into three distinct phases. Initially, the provided audio files, namely *Trumpet.wav* and *Vibraphone.wav*, were read and transformed into a .dat file format. This .dat format encapsulates audio data in a two-column matrix, facilitating subsequent pro- cessing. In the subsequent phase, the adaptive filtering algorithm, originally written in Python, was translated, and adapted into C++ to isolate the trumpet sounds from the vibraphone sounds effectively. Finally, the filtered audio data in .dat format underwent conversion back to the .wav file type.

To streamline the development and compilation process, a *Makefile* was meticulously crafted. This Makefile automates the compilation process, producing a target executable named *'lab2'*. Furthermore, it manages workspace clean up, ensuring an organized environment for iterative code development and project rebuilding.

Python Script

This Python script uses the *Least Mean Squares (LMS)* method to construct a basic adaptive filtering algorithm that eliminates the sound of a trumpet from a vibraphone recording. In order to reduce the discrepancy between the vibraphone recording and the trumpet recording that is played back through the filter, the algorithm learns the impulse response of a *Finite Impulse Response* filter. Using the *'soundfile'* module, the script reads in the two audio files, *'Vibraphone.wav'* and *'Trumpet.wav'*, and stores the sample data in the *'wet'* and *'dry'* variables, respectively. A sample rate of 96 kHz is selected. Additionally, the skip time (ss) is set to 4.9 seconds, ignoring the first 4.9 seconds of the file for the learning phase. The length of the FIR filter is set to 0.3 seconds, or *'fl=0.3'*, which translates to 28,800 samples at a sample rate of 96 kHz. The LMS algorithm's learning rate, *'lrate=0.002'*, is set to 0.002. The sample rate and the filter's length in seconds are used to calculate the number of coefficients for the FIR filter. The script then uses the *'fir1'* module to set up a FIR filter object after calculating the mean squared value of the source signal, which is a vibraphone recording. After that, the LMS algorithm is used to run the training and source signals through the filter for a predetermined number of iterations. Lastly, the script creates a new audio file containing the error signal that is obtained by deducting the FIR filter's output from the original signal. Additionally, the power increase is computed and displayed on the console.

C++ Script

```
#include <iostream>
#include <cmath>
#include <vector>
#include <stdio.h>
#include "sox.h"
#include <algorithm>
#include "Fir1.h"
```

The script incorporates a comprehensive set of essential header files and libraries to ensure its seamless functionality. It includes `<iostream>` for streamlined input and output operations and employs `<cmath>` to facilitate a range of mathematical computations. Additionally, the `<vector>` library is utilized for efficient vector or dynamic array management, while `<sox.h>` is included to harness advanced audio file reading and processing capabilities. To optimize data handling and manipulation, the `<algorithm>` library is incorporated, offering a suite of data structure operations. Furthermore, the custom header `<Fir1.h>` is integrated to access specialized filtering functionalities pivotal to the script's operation. Completing this robust set of libraries, `<stdio.h>` is also enlisted, providing standard input and output functionalities to augment the script's overall capabilities.

```
int main() {
    sox_init();

    const char* in_file_wet = "Vibraphone.wav";
    const char* in_file_dry = "Trumpet.wav";
    const char* out_file = "output.wav";
```

The sox_init() function is called to initialize the Sox library, which will be used for audio processing within the program. Next, the names of the input audio files are declared and initialized. The file 'Vibraphone.wav', representing the original vibraphone recording, is assigned to a constant character pointer named in_file_wet. Similarly, the file 'Trumpet.wav', representing the trumpet recording, is assigned to a constant character pointer named in_file_dry.

Additionally, an output file named ‘output.wav’ is declared and initialized to a variable named out_file. These variables serve as references to the respective audio files throughout the program's execution.

```
sox_signalinfo_t in_signal_wet, in_signal_dry;
sox_format_t *in_file_wet_format = sox_open_read(in_file_wet,
&in_signal_wet, NULL, NULL);
sox_format_t *in_file_dry_format = sox_open_read(in_file_dry,
&in_signal_dry, NULL, NULL);
```

Two sox_signalinfo_t structures, in_signal_wet and in_signal_dry, are declared in this section of the code. Information regarding the audio signals from the input files ‘Trumpet.wav’ and ‘Vibraphone.wav’ will be stored in these structures, respectively.|

Two pointers to sox_format_t are initialised after the structure declarations:

in_file_wet_format: This pointer uses the sox_open_read() function to open and read the ‘Vibraphone.wav’ file. The function first accepts the name of the input file (in_file_wet), then obtains and stores signal information in the in_signal_wet structure. The final two parameters are set to NULL, meaning that no special processing choices or effects are performed while the signal is being read.

in_file_dry_format: Similar to that, this pointer uses the sox_open_read() function to open and read the "Trumpet.wav" file. The function functions similarly to the previous one; it stores the retrieved signal information in the in_signal_dry structure and accepts as its first argument the name of the dry input file (in_file_dry).

```
if (in_file_wet_format == NULL || in_file_dry_format == NULL) {
    std::cerr << "Error opening input file(s).\n";
    return 1;
}
```

Within this section of the code, an *if* statement is employed to validate the successful opening of the two input audio files. The sox_open_read() function returns a NULL pointer if there are any issues accessing the specified files. In the event that either in_file_wet_format or in_file_dry_format is NULL, indicating a failed file read, an error message is displayed to the console using std::cerr. Following the error message, the program is exited with a return value of 1, signalling an abnormal termination due to the file access failure.

```
int sample_rate = in_signal_wet.rate;
int num_channels = in_signal_wet.channels;

std::cout << "Sample rate: " << sample_rate << ", channels: " <<
num_channels << std::endl;

size_t num_samples_wet = in_signal_wet.length / num_channels;
size_t num_samples_dry = in_signal_dry.length / num_channels;
```

By appending ".rate" and ".channels" to the input in_signal_wet (input file name), information about the sample rate and channels of the wet file is obtained. These values are then saved as integer variables called "sample_rate" and "num_channels." Then, "std::cout" is used to print these back to the console. The length of the.wav files and the number of channels are then divided to determine the total number of channels in the wet and dry files. By appending

". length" to the corresponding filenames, one may determine the length of the files. The output is then divided by the variable "num_channels." The corresponding variables, "num_samples_dry" and "num_samples_wet," hold the findings for each.

```
std::vector<float> wet(num_samples_wet), dry(num_samples_dry);

sox_read(in_file_wet_format, wet_sox.data(), num_samples_wet);
sox_read(in_file_dry_format, dry_sox.data(), num_samples_dry);

std::cout << "Wet file. Samples: " << num_samples_wet << ", max/min
normalised samples: " << *std::max_element(wet.begin(), wet.end()) <<
", " << *std::min_element(wet.begin(), wet.end()) << std::endl;

std::cout << "Dry file. Samples: " << num_samples_dry << ", max/min
normalised samples: " << *std::max_element(dry.begin(), dry.end()) <<
", " << *std::min_element(dry.begin(), dry.end()) << std::endl;
```

Two instances of the C++ std::vector<float> class, named 'wet' and 'dry', are declared and initialized. The sizes of these vectors correspond to the number of samples in the respective audio files, represented by 'num_samples_wet' and 'num_samples_dry'. The sox_read() function is utilized to populate these vectors with audio data. It takes three arguments:

1. The pointer to the sox_format_t structure corresponding to the input audio file, i.e., 'in_file_wet_format' or 'in_file_dry_format'.
2. The location in memory where the audio samples should be stored, accessed using the data() member function of the vectors ('wet.data()' for 'wet' and 'dry.data()' for 'dry').
3. The number of samples to read from the audio file, denoted by 'num_samples_wet' for the wet file and 'num_samples_dry' for the dry file.

After reading the audio samples into the 'wet' and 'dry' vectors, the program outputs information about the audio files to the console. Specifically, it displays the number of samples in each file and the maximum and minimum normalized sample values found within each vector using std::max_element() and std::min_element().

```
size_t filter_length = 0.3 * sample_rate;
float learning_rate = 0.002;
float skip_seconds = 4.9;

Fir1 fir(filter_length);
fir.setLearningRate(learning_rate);
```

In this segment of the code, three variables are initialized to configure foundational parameters for the FIR1 filter. The filter_length variable, of type size_t, determines the number of taps or coefficients in the FIR filter. It is calculated by multiplying the sample rate by 0.3, representing 30% of the sample rate, and thus indicating the desired length of the FIR filter in terms of sample points. The learning_rate variable, a float, controls the adaptation speed of the FIR filter and is set to a value of 0.002. This value influences the rate at which the filter adjusts its coefficients during the filtering process. Lastly, the skip_seconds variable, also a float, specifies the duration of audio that should be skipped before applying subsequent filtration processes, set here to 4.9 seconds. Following these initializations, an instance of the FIR1 filter, named 'fir', is instantiated. The filter's length is set using the previously defined filter_length, and its learning rate is established through the setLearningRate() method of the FIR1 filter object, with the learning_rate variable serving as its argument.

```
size_t num_samples_skip = skip_seconds * sample_rate;
std::vector<float> training(dry.begin()+num_samples_skip,dry.end());
std::vector<float> src(wet.begin() + num_samples_skip, wet.end());
```

In this section of the code, the exact number of samples to be skipped is determined by multiplying the skip_seconds variable by the sample_rate, resulting in the num_samples_skip value. Two new vectors are then created based on the original 'dry' and 'wet' vectors, excluding the samples corresponding to the skipped duration:

1. The training vector is initialized to store the samples from the 'dry' audio file. It begins from the sample immediately following the skipped duration (i.e., dry.begin() + num_samples_skip) and extends to the end of the 'dry' vector.
2. Similarly, the src vector is created to store the samples from the 'wet' audio file. It starts from the sample following the skipped duration (i.e., wet.begin() + num_samples_skip) and continues to the end of the 'wet' vector.

This approach effectively excludes the specified initial duration from both audio files, preparing the data for subsequent processing without the skipped portions.

```
float msi = 0;
for (auto s : src) {
msi += s * s;
}
msi /= src.size();
```

The initial value of a new variable named ‘msi’ is set to 0. Next, each member of the ‘src’ vector is copied into a variable using a ‘for’ loop. In the following line, this variable is squared, and the result is added to the ‘msi’ variable. Consequently, the cumulative sum of squares of all the wet file audio samples in ‘src’ is the final value of ‘msi’. Eventually, ‘msi’ holds the mean square value of the wet audio samples after being divided by the size of the ‘src’ vector.

```
    if (training.size() != src.size()) {
        std::cerr << "Interference and source samples are different
lengths.\n";
        return 1;
    }
```

The provided code snippet conducts a fundamental verification to confirm that the training vector, which represents the dry samples, and the src vector, representing the wet samples, possess consistent dimensions. The sizes of both vectors are determined by appending .size() to each respective vector name, and their equality is assessed using the inequality operator !=. Should there be a disparity in their sizes, an error message is promptly generated via std::cerr, signalling that the interference and source samples exhibit mismatched lengths. Subsequently, the program terminates with an exit status of 1.

```
size_t trial = 2 * sample_rate;

std::cout << "Pre-training...\n";
for (size_t i = 0; i < std::min(trial, src.size()); i++) {
    fir.lms_update(src[i] - fir.filter(training[i]));
}
```

The provided code snippet initiates a training phase for the FIR filter using a subset of the audio data. A variable named trial is declared and assigned a value of 2 * sample_rate, setting the duration of the trial period to 2 seconds based on the sample rate. A descriptive message, ‘Pre-training...’, is displayed on the console to inform the user about the current stage of the program.

A for loop is employed to iterate through the audio samples. The loop iterates from i = 0 to the minimum value between trial and the size of the src vector, ensuring that the loop does not exceed the bounds of the src vector. Within the loop, the fir.filter(training[i]) method processes each sample from the training vector using the FIR filter. Subsequently, the difference between the filtered output and the corresponding sample from the src vector is computed. This difference, representing the error, is then fed back into the FIR filter using the lms_update method to facilitate the filter's adaptation or training process.

```
std::cout << "Processing...\n";
std::vector<float> error(training.size());
for (size_t i = 0; i < src.size(); i++) {
      float filtered = fir.filter(training[i]);
      error[i] = src[i] - filtered;
      fir.lms_update(error[i]);
}
```

In this code segment, the input signal is processed through the FIR filter, and the error between the filtered dry signal and the wet signal is calculated and stored for subsequent analysis. A console message, "Processing...", is displayed to indicate the current processing stage to the user.

A new vector named error is initialized with a size matching that of the training vector. Within a for loop iterating over the samples of the src vector, each sample from the training vector is filtered using the fir.filter() method, and the resulting filtered value is stored in a variable named filtered. The difference between the original sample from the src vector and the filtered value is computed and stored as a new element in the error vector.

Subsequently, the FIR filter is updated using the lms_update method, with the error value from the error vector serving as feedback to facilitate the adaptive learning or adjustment of the filter.

```
float power_gain = 0;
for (auto e : error) {
    power_gain += e * e;
}
power_gain /= msi;
std::cout << "Power gain: " << power_gain << std::endl;
```

In this code segment, the power gain of the error signal is computed and displayed. A floating-point variable named power_gain is initialized with an initial value of 0. A for loop iterates through each element in the error vector, and each element (e) is squared and added to the power_gain variable.

After the loop concludes, the accumulated power_gain is normalized by dividing it by the mean square of the wet signal, represented by the variable msi. The resulting normalized power gain is then displayed on the console using std::cout.

```
sox_signalinfo_t out_signal;
out_signal.rate = sample_rate;
out_signal.channels = num_channels;
out_signal.precision = SOX_SAMPLE_PRECISION;
out_signal.length = error.size() * num_channels;
out_signal.mult = NULL;
```

The provided code snippet initializes a sox_signalinfo_t structure named out_signal, which is used to define the properties of the output audio signal to be generated or processed by the program. The out_signal.rate is set to sample_rate, aligning the output signal's sample rate with that of the input signals. Similarly, out_signal.channels is assigned the value stored in num_channels, determining the number of audio channels in the output, whether it's mono, stereo, or multi-channel. The out_signal.precision is set to a standard value, SOX_SAMPLE_PRECISION, ensuring a consistent and appropriate bit-depth for the output audio. To determine the total number of samples in the output signal, out_signal.length is calculated by multiplying the size of the error vector with num_channels, providing the overall length considering the number of channels. Lastly, the out_signal.mult field is initialized to NULL, indicating that no specific scaling or multiplier is applied to the signal at this stage. Overall, these lines of code prepare the out_signal structure with the necessary parameters to correctly format and generate the output audio signal based on the provided input and error data.

```
sox_format_t *out_file_format = sox_open_write(out_file, &out_signal,
NULL, "wav", NULL, NULL);

if (out_file_format == NULL) {
    std::cerr << "Error opening output file.\n";
    return 1;
}
```

In order to ensure that the file was generated successfully, these lines first open and write the output file. The ‘sox_format_t’ structure is pointed to by a pointer called ‘out_file_format’, which is generated. The ‘sox_open_write’ function is used to open and write to the output file. It requires the following six arguments: ‘out_file’ (the file name), ‘&out_signal’ (a pointer to the sox_signinfo_t' structure), NULL (for encoding functions), ‘wav’ (the file format), NULL (format settings), and NULL (other info).

```
sox_close(out_file_format);
sox_close(in_file_wet_format);
sox_close(in_file_dry_format);

sox_quit();

return 0;
}
```

In order to write processed error data into the output file, the'sox_write' function is used first. Following that, all three files—two inputs and one output—are closed using the'sox_close' function. The SoX library is terminated using "sox_quit()," and a value of 0 is returned to signify successful execution.

Conclusion

In conclusion, both C++ and Python offer viable options for implementing adaptive filtering algorithms, each with distinct advantages. C++ is a compiled language known for its performance and efficiency, making it well-suited for computationally intensive tasks and real-time processing. Its low-level nature provides developers with precise control over system resources.

On the other hand, Python, as an interpreted language, excels in readability and ease of use. It facilitates rapid development and prototyping due to its high-level abstractions and concise syntax. Additionally, Python's extensive ecosystem of libraries in data analysis, scientific computing, and machine learning enhances its suitability for projects requiring complex data manipulation. The choice between C++ and Python should be driven by the project's specific requirements, such as performance needs, development timelines, and available libraries. While C++ may be preferable for performance-critical applications, Python is often more appropriate for projects emphasizing rapid development and code clarity. Ultimately, the decision should align with the project's goals, ensuring a balance between performance, development efficiency, and maintainability.

Appendix

C++ CODE

```
#include <iostream>
#include <cmath>
#include <vector>
#include <stdio.h>
#include "sox.h"
#include <algorithm>
#include "Fir1.h"

int main() {
    sox_init();

    const char* in_file_wet = "Vibraphone.wav";
    const char* in_file_dry = "Trumpet.wav";
    const char* out_file = "output.wav";

    sox_signalinfo_t in_signal_wet, in_signal_dry;
    sox_format_t  *in_file_wet_format  =  sox_open_read(in_file_wet,
&in_signal_wet, NULL, NULL);
    sox_format_t  *in_file_dry_format  =  sox_open_read(in_file_dry,
&in_signal_dry, NULL, NULL);

    if (in_file_wet_format == NULL || in_file_dry_format == NULL) {
        std::cerr << "Error opening input file(s).\n";
        return 1;
    }

    int sample_rate = in_signal_wet.rate;
    int num_channels = in_signal_wet.channels;

    std::cout << "Sample rate: " << sample_rate << ", channels: " <<
num_channels << std::endl;

    size_t num_samples_wet = in_signal_wet.length / num_channels;
    size_t num_samples_dry = in_signal_dry.length / num_channels;

    std::vector<float> wet(num_samples_wet), dry(num_samples_dry);

    std::vector<sox_sample_t> wet_sox(num_samples_wet);
    std::vector<sox_sample_t> dry_sox(num_samples_dry);

    sox_read(in_file_wet_format, wet_sox.data(), num_samples_wet);
    sox_read(in_file_dry_format, dry_sox.data(), num_samples_dry);

    std::cout  <<  "Wet  file.  Samples:  "  <<  num_samples_wet  <<  ",
max/min  normalised  samples:  "  <<  *std::max_element(wet.begin(),
wet.end())  <<  ",  "  <<  *std::min_element(wet.begin(),  wet.end())  <<
std::endl;
    std::cout  <<  "Dry  file.  Samples:  "  <<  num_samples_dry  <<  ",
max/min  normalised  samples:  "  <<  *std::max_element(dry.begin(),
dry.end())  <<  ",  "  <<  *std::min_element(dry.begin(),  dry.end())  <<
std::endl;
```

```
    for (size_t i = 0; i < num_samples_wet; i++) {
        wet[i]  =  SOX_SAMPLE_TO_FLOAT_32BIT(wet_sox[i],  sox_glob-
als.clip);
        }

    for (size_t i = 0; i < num_samples_dry; i++) {
        dry[i]  =  SOX_SAMPLE_TO_FLOAT_32BIT(dry_sox[i],  sox_glob-
als.clip);
        }

    size_t filter_length = 0.3 * sample_rate;
    float learning_rate = 0.002;
    float skip_seconds = 4.9;

    Fir1 fir(filter_length);
    fir.setLearningRate(learning_rate);

    size_t num_samples_skip = skip_seconds * sample_rate;
    std::vector<float>   training(dry.begin()   +   num_samples_skip,
dry.end());
    std::vector<float>     src(wet.begin()     +     num_samples_skip,
wet.end());

    float msi = 0;
    for (auto s : src) {
        msi += s * s;
    }
    msi /= src.size();

    if (training.size() != src.size()) {
        std::cerr << "Interference and source samples are different
lengths.\n";
        return 1;
    }

    // Run a couple of seconds through the filter so it's started to
learn.
    size_t trial = 2 * sample_rate;

    std::cout << "Pre-training...\n";
    for (size_t i = 0; i < std::min(trial, src.size()); i++) {
        fir.lms_update(src[i] - fir.filter(training[i]));
    }

    std::cout << "Processing...\n";
    std::vector<float> error(training.size());
    for (size_t i = 0; i < src.size(); i++) {
        float filtered = fir.filter(training[i]);
        error[i] = src[i] - filtered;
        fir.lms_update(error[i]);
}

float power_gain = 0;
for (auto e : error) {
    power_gain += e * e;
```

```
}
power_gain /= msi;
std::cout << "Power gain: " << power_gain << std::endl;

sox_signalinfo_t out_signal;
out_signal.rate = sample_rate;
out_signal.channels = num_channels;
out_signal.precision = SOX_SAMPLE_PRECISION;
out_signal.length = error.size() * num_channels;
out_signal.mult = NULL;

sox_format_t *out_file_format = sox_open_write(out_file, &out_signal,
NULL, "wav", NULL, NULL);

if (out_file_format == NULL) {
    std::cerr << "Error opening output file.\n";
    return 1;
}


sox_close(out_file_format);
sox_close(in_file_wet_format);
sox_close(in_file_dry_format);

sox_quit();

return 0;
}
```

MAKEFILE

```
Lab2: audioio.o main.cpp libfir_static.a
      g++ -g main.cpp -o lab2 audioio.o -L. -
lfir-static

audioio.o: audioio.cxx
      g++ -c -g audioio.cxx

CXXFLAGS += -I./fir1
```